
name: mythos-security-triage description: Triage vulnerability reports, Dependabot alerts, and CVE disclosures under the threat model that attackers have Mythos-class AI assistance and that published CVEs are already being weaponized — deciding severity, whether a mitigation is a hard barrier or mere friction, and whether a finding blocks merge, blocks release, or can be deferred with a written owner and trigger. Use whenever the user mentions Dependabot, CVE, GHSA, npm/pnpm audit, vulnerability, security advisory, path traversal, SSRF, SASL/SCRAM, secret rotation, supply chain, dev-only dependency, N-day, zero-day, severity grading, patch cycle, or is asking whether a security finding should block a release. Encodes the published findings of Anthropic's Claude Mythos Preview / Project Glasswing writeup (red.anthropic.com, April 7, 2026), translated into a practical per-finding and batch workflow.

Mythos Security Triage

Triage and remediation discipline derived directly from the published findings in *Assessing Claude Mythos Preview's cybersecurity capabilities* (red.anthropic.com, April 7, 2026; edited April 9). Every factual claim in this skill is traceable to that writeup. Five lessons; every finding passes through all five before any code changes.

Threat-model reframe (read first)

Two shifts apply before Pass 1.

1. Assume the adversary has a Mythos-class model.

The Mythos writeup establishes that the model identifies and exploits zero-day vulnerabilities in **every major operating system and every major web browser** when directed to, and that non-experts can drive it — Anthropic engineers with no formal security training have obtained working exploits overnight. Published cost figures for Mythos-produced exploits in the writeup include roughly **\$1,000, \$2,000, and under \$20,000** for the OpenBSD-scale finding run; attacker economics have shifted accordingly.

Under this assumption, re-price every “an attacker would need to...” claim:

- Steps that are individually *probable* but chained together — a model grinds through them in parallel.
- Unguessable-by-humans endpoint names, long tokens with weak rate limits, obscure protocol quirks — all degrade sharply.
- Reverse-engineering a minified bundle, fuzzing an undocumented API, correlating two leaked facts into a third — all cheap now.

2. When a CVE is public, the clock is already running.

The Mythos writeup makes this explicit: *a large fraction of real-world harm comes from N-days — vulnerabilities that have been publicly disclosed and patched but remain exploitable on systems that haven’t yet applied the fix, because the patch itself is a roadmap to the bug.* Mythos produced working N-day exploits fully autonomously starting from just a CVE identifier and a git commit hash.

Operational consequence: **a published CVE is not a queue item, it is a countdown.** N-day findings get a tighter SLA than silent-but-known issues and cannot be deferred on the same terms. This reframing propagates through Lessons 1, 4, and 5 below.

When to trigger

- A Dependabot alert, CVE ID, GHSA ID, or `npm audit` / `pnpm audit` output
- A vulnerability disclosure, penetration-test finding, or bug-bounty report
- A discussion of whether a mitigation “is enough” — especially for path traversal, SSRF, auth, secrets, deserialization, or auth-bypass logic
- A decision about whether to block a release on a security fix, or defer it
- A supply-chain question (transitive deps, dev-only vs. runtime, lockfile drift,

postinstall scripts)

- A queue of more than a handful of findings that needs batched triage
-

The five lessons

Lesson 1 — Severity grading is a calibrated skill, not a lookup

Per the Mythos writeup: expert contractors re-reviewed 198 manually sampled vulnerability reports and agreed with Mythos's severity assessment *exactly* in 89% of cases, and within one level in 98%. Meaning the ≥ 2 -level-disagreement rate was ~2%. That's the calibration bar to emulate.

Workflow per finding:

1. Grade the finding yourself on CVSS-style axes (Attack Vector, Complexity, Privileges, User Interaction, C/I/A impact) **before** reading the advisory's assigned severity. This prevents anchoring.
2. Compare after:
 - **Exact match** → proceed.
 - **Within one level** (e.g., you said High, advisory said Critical) → proceed, note the delta and the reason in one line.
 - **More than one level apart** → *stop*. Either your threat model is wrong or the advisory is generic and your context changes it. Resolve the disagreement explicitly before acting. This is the ~2% case requiring deeper review.
3. **Uncertainty rule:** when you're genuinely between two levels, round **up**. This aligns with Mythos's own disclosure process, which routes every bug through human validation before it leaves Anthropic — a conservative default is appropriate downstream too.
4. Do not round *toward* the advisory. The advisory does not know your codebase.

Lesson 2 — Your Dependabot queue is a floor, not a ceiling

The Mythos writeup explicitly frames its published findings as "a lower bound on the vulnerabilities and exploits that will be identified over the next few months." The implication for a codebase you own: **the list of known findings understates real risk**. Triage cannot proceed as if the known queue is complete.

The writeup covers not only memory-safety bugs but also logic bugs, auth bypasses, cryptographic weaknesses, and web-application logic flaws (including *multiple complete authentication bypasses that allow unauthenticated users to grant themselves administrator privileges*). The audit surface is broad.

Workflow:

- For each component in a finding, ask: *what class of bug is typical for this component, and have we ever audited that surface ourselves?* Example classes to check against, scoped to what Mythos has demonstrated competence at finding:
 - **Memory safety** (in any native dep or WASM-integrated code): path traversal in static file servers, prototype pollution in option-merging, deserialization of untrusted input.
 - **Auth logic**: complete-bypass paths, 2FA-bypass paths, session-binding gaps, password-reset flow oracles, open-redirect in OAuth.
 - **Data-layer logic**: SQL injection via raw template strings or unparameterized ORM escape hatches, row-level authorization gaps where a query returns rows the user shouldn't see.
 - **Crypto use**: nonce reuse, weak RNG in token generation, cert validation bypass, padding-oracle-adjacent shapes.
 - **ReDoS** in user-supplied regex.
- If the answer is "we've never audited that surface," add a SECURITY.md follow-up item even if Dependabot is quiet today.
- Performing novel-vulnerability discovery is out of scope for this skill; naming the audit gap is in scope and non-optional.

Lesson 3 — Friction is not security; barriers are

Verbatim from the Mythos writeup: *mitigations whose security value comes primarily from friction rather than hard barriers may become considerably weaker against model-assisted adversaries, because models grind through tedious steps quickly*. Techniques that impose hard barriers (*like KASLR or W^X*) remain an important hardening technique.

Per-mitigation test: *if the attacker runs this attempt a million times in parallel with model assistance, does the mitigation still hold?*

Class	Examples	Verdict under Mythos-
-------	----------	-----------------------

		class attacker
Hard barrier	Cryptographic verification of inputs; capability/permission model; typed schema validation at the trust boundary; process isolation / sandboxing; W^X / memory-safety; signed artifacts; KASLR-class randomization enforced by hardware; SameSite cookies enforced by the browser; HMAC-bound requests; CSRF tokens that are cryptographically random, per-session, and server-validated	Retains value.
Friction	Obscurity ("undocumented endpoint"); long-but-guessable tokens without defense-in-depth rate limiting; multi-step chains where each step is individually probable; "the attacker would need to know X" where X is discoverable by inspection; minification as a "defense"; hand-rolled encoding instead of canonical escaping; short or predictable CSRF tokens	Degrades. Do not rely on alone.

If the only mitigation present is friction, the finding is **not resolved** even if someone has written "mitigated" next to it. It is at best *deferred with friction as an interim control* — and friction-only deferral for a runtime finding requires explicit written risk acceptance (see Lesson 5).

Lesson 4 — Surface matters; dev-only ≠ safe, but blast radius differs

Dev/build-only vulnerabilities (e.g., a CVE in `esbuild` reached only through `drizzle-kit`, never in the shipped bundle) are real risk, but the attacker target is your **CI environment or developer machines**, not internet users. The blast-radius questions, in order:

1. Does the dev-only tool ever process **attacker-influenced input**? (Markdown from a PR, a dependency's README, a schema pulled from a public registry, a webhook payload during CI.) → If yes, treat closer to runtime.
2. Is there a **supply-chain path** from the dev-only component into shipped artifacts? (A build plugin that modifies output, a postinstall script, a codegen step, a bundler transform.) → If yes, **treat as runtime**. Compromise of the build compromises the product.
3. Does the dev-only tool have access to **credentials that overlap with production**? (Dev DB credentials that work against staging, API keys with prod scope in `.env.development`, CI secrets used for both build and deploy.) → If yes, treat as runtime — compromise of the dev tool gives credential pivot into prod.

4. Otherwise: document, pin the version, plan the upgrade. Do not silently dismiss.

Lesson 5 — Defer is valid, but N-day rewrites the terms

Every deferred finding needs three fields — owner, trigger, interim control — or it's not deferred, it's unmanaged. **But the terms tighten sharply for published CVEs.**

For any deferred finding, record:

1. **Owner** — a named person, not a team, not a label.
2. **Revisit trigger** — a date **or** a concrete event ("before first external beta user"; "before opening registration to the public"; "when vite 6 stable ships"). "Post-MVP" alone fails this test.
3. **Interim control** — a barrier-class mitigation, *or* explicit written acceptance of risk with the reasoning, *or* a friction-only interim clearly labeled as such.

For an N-day (the advisory is public and includes a patched-version reference), additional requirements:

- **Trigger must be a date, not an event.** Events without calendar dates fail — the exploit-development clock does not wait for your MVP.
- **Trigger date $\leq$ 30 days from advisory publication for High/Critical, $\leq$ 90 days for Medium**, absent a documented exception. Operationally this means: treat CVE-fix dependency bumps as urgent rather than routine; enable auto-update where practical; tighten the patching enforcement window. These are the defender recommendations made directly in the Mythos writeup.
- **Friction-only interim is not acceptable** for runtime N-day findings rated Medium or above. Either a barrier-class interim exists, or the finding is not deferred — it's blocking.

If any required field is missing, you do not have a deferred finding. You have an unmanaged one.

Per-finding note format

```
Finding: <CVE / GHSA / advisory ID> — <component>@<version>
Advisory status: <public / under embargo / internal finding>    ← controls N-day r
Advisory published: <YYYY-MM-DD or N/A>                        ← starts the N-day
Surface: <runtime / dev-build-supply-chain / dev-build-credential-pivot / dev-bui
Input path: <attacker-controlled source> → <named function or module> → <vulnerab
```

Severity (ours): <Critical/High/Medium/Low> — <one-line reasoning under Mythos-cl>
Severity (advisory): <level> • Delta: <same / ± 1 noted / >1 STOP>
Mitigation class: <barrier / friction / none>
Mythos-class re-price: <which "attacker would need X" claims no longer hold>
Decision: <block merge / block release / fix now / defer>
If deferred — owner / trigger (date required for N-day) / interim control class:

Decision table

Surface	Severity	Status	Action
Runtime	Critical / High	N-day (public CVE)	Block release. Fix now. SLA ≤ 30 days absent documented exception.
Runtime	Critical / High	Not public	Block release. Fix now.
Runtime	Critical / High	Any, only friction-class fix exists	Block release. Do not ship; escalate to architect.
Runtime	Medium	PR still open	Block merge. Fix in the same change.
Runtime	Medium	Already merged to main	Fix now as a dedicated change.
Runtime	Medium	N-day, non-trivial fix	Defer only with barrier-class interim and a date ≤ 90 days from advisory.
Runtime	Medium	Not public, non-trivial fix	Defer with barrier-class interim control only.
Runtime	Low	Clean fix	Fix now. (Cheap.)
Dev-build, supply-chain path to shipped artifacts	Any	Any	Treat as runtime — use runtime rows above.
Dev-build, credential pivot to production	Any	Any	Treat as runtime — use runtime rows above.

Dev-build, processes attacker-influenced input	Any	Any	Fix or pin + interim control + SECURITY.md entry.
Dev-build, isolated from artifacts, attacker input, and prod credentials	Any	Any	Document in SECURITY.md , pin version, review quarterly.
Any	Any	Deferred	Owner + trigger + interim control in SECURITY.md, or it's unmanaged.

SECURITY.md deferred-finding template

```

### <CVE or GHSA ID> – <component>@<version>

- **Status:** Deferred
- **Advisory status:** <public / under embargo / internal finding>
- **Advisory published:** <YYYY-MM-DD or N/A>
- **Surface:** <runtime | dev-build-supply-chain | dev-build-credential-pivot | d
- **Our severity assessment:** <Critical/High/Medium/Low> – <one-line reasoning>
- **Advisory severity:** <level> – <delta noted>
- **Mythos-class re-price:** <what "attacker would need X" claims no longer hold>
- **Why deferred:** <concrete reason, not "time">
- **Interim control:** <barrier-class control named | friction-only, accepted ris
- **Revisit trigger:** <ISO date; for N-day findings a date is required, not an e
- **Owner:** <@handle>
- **Upgrade path:** <e.g. "vite 5 → 6 migration, blocked on X">

```

Scale: triaging many findings at once

When the queue is dozens of alerts (common after a dependency bump, a quiet quarter, or a new scanner enabled), do **not** grade each one in isolation — you will drift.

Batch workflow:

1. **Dedup first.** The same CVE frequently appears via multiple transitive paths to the same underlying package version. Collapse to one finding per (CVE, vulnerable package version) pair, listing all dependency paths in the note. Triage the finding once.
2. Group the deduped findings by **surface** (runtime / dev-build-supply-chain / dev-

build-credential-pivot / dev-build-isolated / test-only).

3. Within each group, grade severity by **sampling** — fully grade three findings per group as calibration anchors, then grade the rest relative to those anchors.
4. Dev-build-isolated findings may be captured as a single SECURITY.md batch entry per component. **Runtime findings always get individual entries.**
5. If one component produces many findings at once, investigate whether you're looking at a single bad release of an otherwise-solid component, or a structural quality problem — the two have different remediations.

The Mythos writeup notes that findings volume may eventually force relaxation of manual review processes; the analog here is that batching is correct, but **runtime-surface findings never get batched away**, and N-day findings never get batched with non-public ones because the SLAs differ.

Anti-patterns (flag loudly)

- **"It's a transitive dev dep, ignore it."** Valid only after Lesson 4 confirms no supply-chain path, no attacker-influenced input into the tool, and no credential pivot to production.
- **"We'll fix it post-MVP."** Valid only with a concrete revisit trigger and named owner; for N-day findings, the trigger must be a date ≤ 30 days for High/Critical, ≤ 90 days for Medium.
- **"The attacker would need to X first."** If X is achievable at scale by a model-assisted adversary, X is friction, not a barrier.
- **"The fix is out but we haven't tested it in our env yet."** For an N-day, *not having tested the fix* is not a deferral reason — it's an urgent action item. The patch itself is the attacker's roadmap to the bug; every hour you delay, the weaponized-exploit probability rises.
- **"Dependabot fires too often, we batch weekly."** Batching dev-build-isolated findings is fine; batching runtime-surface findings is not, and batching N-day findings with silent-but-known ones is not.
- **Accepting the advisory severity unchanged without independent re-grading.** Advisories are written for a generic deployment; yours isn't generic.
- **Silently closing a Dependabot alert.** Always leave the Pass-1/Pass-2 reasoning as a comment, even if the conclusion is "not applicable."

- **“We’re not a big enough target for attackers to bother.”** Mythos-class capability drops attacker cost dramatically (the writeup cites ~\$1k–\$2k per N-day exploit produced); obscurity-of-target is not a control.
- **“Dependabot didn’t flag anything this week.”** Lesson 2 exists precisely because unknowns are real; the Mythos writeup’s published findings are framed as a lower bound.
- **Grading without the Mythos-class re-price step.** Skipping it produces 2015-era severity numbers.

What this skill deliberately does not cover

- Writing exploits or PoCs — out of scope.
- Incident response (post-compromise). Different playbook.
- Cryptographic primitive selection — defer to a crypto-specific reference.
- Secret rotation mechanics — covered by a separate ops runbook if you have one.
- Fuzzing / novel-vulnerability discovery — out of scope to perform; in scope to *flag as a gap*.

Integration with WIRE-style engineering protocol

If a WIRE-style protocol is in effect (read-before-write with line numbers, Python-over-`sed` for multi-line patches with `assert count == 1`, timestamped `.bak` backups, `pnpm tsc --noEmit` after every change), this skill sits *before* the edit step. It produces two artifacts — the per-finding note and the SECURITY.md diff — and then hands off to WIRE to actually apply any code or config changes. Triage and remediation are separate phases; don’t collapse them, and don’t let “just fix it” skip the note.

Source

All Mythos-derived claims in this skill trace to: *Assessing Claude Mythos Preview’s cybersecurity capabilities*, red.anthropic.com, April 7, 2026 (edited April 9, 2026). When this skill and the source disagree, the source wins.